MML â€” Mini Math Language

CS315 Programming Languages â€” Term Project Report

---

1. Introduction

MML (Mini Math Language) is a domain-specific scripting language designed for mathematical computation. It supports variables, arithmetic expressions, boolean conditions, three loop constructs, user-defined functions, and nine built-in math functions. The implementation follows the classic compiler front-end pipeline: a lexer generated by **Flex**, a parser generated by **Bison (Yacc)**, and a hand-written tree-walking executor in C.

Why MML?

The reference example (DQL) is a *declarative* data-query language for tabular data (LOAD / FILTER / JOIN / SHOW). MML is deliberately *imperative*: it has mutable variables, control flow (IF, WHILE, FOR), and callable functions â€” concepts that require a fundamentally different grammar, AST design, and runtime model.

Feature	DQL (reference)	MML (this project)
Style	Declarative	Imperative
Variables	None	Yes (SET)
Control flow	None	IF / WHILE / FOR
Functions	None	DEF (user-defined)
I/O	SHOW	PRINT
Data domain	CSV / JSON tables	Floating-point math
Built-ins	None	SIN, COS, TAN, SQRT, ABS, LOG, EXP, FLOOR, CEIL

---

2. Language Design

2.1 Design Goals

- **Readable**: Keywords are English words (SET, PRINT, IF, THEN, ELSE, END, WHILE, FOR, DEF).
- **Mathematical**: All values are IEEE 754 doubles; division is always floating-point; power (^) is a first-class operator.
- **Case-insensitive**: Identifiers and keywords are matched without regard to case (e.g., `sin`, `SIN`, `Sin` are all the same).
- **Simple scoping**: One global environment; function calls bind parameters into a local copy of that environment.

2.2 Data Types

MML has exactly one data type: **double-precision floating-point** (`double`). There are no integers, strings (as values), or booleans â€” conditions evaluate to `int` (0 or 1) internally but are not first-class values.

2.3 Statement Types

Statement	Syntax	Description
Variable assignment	`SET x = expr`	Evaluate `expr`, store in `x`
Print	`PRINT e1 [, e2 ...]`	Print space-separated values; strings are literals
Conditional	`IF cond THEN stmts [ELSE stmts] END`	Branching
While loop	`WHILE cond DO stmts END`	Condition-controlled loop
For loop	`FOR v FROM e1 TO e2 [STEP e3] DO stmts END`	Counted loop; step defaults to +1
Function def	`DEF f(p1, ...) = expr`	Single-expression user function

2.4 Operators

Category	Symbols	Associativity
Logical OR	`OR`	left
Logical AND	`AND`	left
Logical NOT	`NOT`	right (unary)
Comparison	`== != < <= > >=`	non-associative
Addition	`+ -`	left
Multiplication	`* / %`	left

Power	^	right
Unary minus	-	right (highest)

2.5 Built-in Functions

`SIN`, `COS`, `TAN`, `SQRT`, `ABS`, `LOG`, `EXP`, `FLOOR`, `CEIL` â€” all take exactly one argument.

---

3. Context-Free Grammar (BNF)

```
program     ::= stmt_list | Îµ

stmt_list   ::= stmt
              | stmt_list stmt

stmt        ::= SET IDENT '=' expr
              | PRINT print_args
              | IF cond THEN stmt_list END
              | IF cond THEN stmt_list ELSE stmt_list END
              | WHILE cond DO stmt_list END
              | FOR IDENT FROM expr TO expr DO stmt_list END
              | FOR IDENT FROM expr TO expr STEP expr DO stmt_list END
              | DEF IDENT '(' param_list ')' '=' expr

param_list  ::= IDENT
              | param_list ',' IDENT

print_args  ::= print_item
              | print_args ',' print_item

print_item  ::= expr | STRING

cond        ::= expr CMP expr
              | cond AND cond
              | cond OR cond
              | NOT cond
              | '(' cond ')'

expr        ::= NUMBER
              | IDENT
              | expr '+' expr
              | expr '-' expr
              | expr '*' expr
              | expr '/' expr
              | expr '%' expr
              | expr '^' expr
              | '-' expr          (unary minus)
              | '(' expr ')'
              | BUILTIN '(' expr ')'
              | IDENT '(' call_args ')'

call_args   ::= expr
              | call_args ',' expr
```

The grammar is LALR(1). Operator precedence and associativity are specified via Bisonâ€™s `%left`/`%right`/`%nonassoc` directives rather than by grammar stratification.

---

4. Lexer Design (mml.l)

The lexer is written in Flex with the following options:

```
%option case-insensitive   // all pattern matching ignores case
%option noyywrap           // no multi-file input
%option nounput noinput    // suppress unused function warnings
```

4.1 Token Categories

Category	Examples	Token returned
Keywords	`SET`, `PRINT`, `IF`, â€¦	Dedicated token (e.g., `SET`)
Built-in names	`SIN`, `COS`, `SQRT`, â€¦	`BUILTIN` (yylval.bfunc set)
Comparison ops	`==`, `!=`, `<=`, â€¦	`CMP` (yylval.cmp set)
Arithmetic ops	`+`, `-`, `*`, `/`, `%`, `^`	Single-char token
Numeric literal	`42`, `3.14`, `.5`	`NUMBER` (yylval.num = atof)
String literal	`"hello"`	`STRING` (yylval.str, no quotes)
Identifier	`x`, `myVar`	`IDENT` (yylval.str = xstrdup)

4.2 Pattern Definitions

```
ALPHA   [A-Za-z_]
ALNUM   [A-Za-z0-9_]
DIGIT   [0-9]
INTEGER {DIGIT}+
FLOAT   {DIGIT}+"."{DIGIT}*|"."{DIGIT}+
NUMBER  {FLOAT}|{INTEGER}
```

4.3 Design Note: Keyword vs.Â Identifier Clash

Because `%option case-insensitive` is active, **identifiers must not match keyword or built-in names**. This caused one bug during development: using `exp` as a loop variable in a sample program clashed with the `EXP` built-in token, producing a parse error. The fix was to rename the variable to `pw` (power).

---

5. Parser Design (mml.y)

The parser uses Bisonâ€™s LALR(1) algorithm. The `%union` directive declares five value types:

```
%union {
    double       num;    // NUMBER literal
    char        *str;    // IDENT / STRING
    int          cmp;    // CmpOp enum
    int          bfunc;  // BuiltinFunc enum
    ExprNode    *expr;   // expression AST node
    CondNode    *cond;   // condition AST node
    StmtNode    *stmt;   // statement / list
    PrintItem   *pitem;  // PRINT argument list
}
```

5.1 Operator Precedence

Declared lowest-to-highest:

```
%left  OR
%left  AND
%right NOT
%nonassoc CMP
%left  '+' '-'
%left  '*' '/' '%'
%right '^'
%right UMINUS
```

`UMINUS` is a pseudo-token used to give unary minus higher precedence than binary subtraction via `%prec UMINUS`.

5.2 Semantic Actions

Each grammar rule builds an AST node using constructor functions defined in `executor.c`. For example:

```
stmt : SET IDENT '=' expr   { $$ = make_stmt_set($2, $4); }
     | PRINT print_args      { $$ = make_stmt_print($2); }
     | FOR IDENT FROM expr TO expr STEP expr DO stmt_list END
                              { $$ = make_stmt_for($2, $4, $6, $8, $10); }
```

The top-level action calls `exec_program($1)` immediately after a successful parse.

5.3 Function Call Argument Parsing

User-defined function calls (`IDENT '(' call_args ')'`) use a global `arg_buf[MAX_ARGS]` / `arg_count` pair populated by the `call_args` rule. This is a simplification: nested calls where the outer callâ€™s argument is itself a call work correctly because inner call_args actions fire (and overwrite arg_buf) before the outer IDENT action fires.

---

6. Runtime / Executor (executor.c)

The executor is a recursive tree walker that traverses the AST produced by the parser.

6.1 Environment

```
typedef struct {
    SymbolTable symbols;   // variable store
    FuncTable   funcs;     // user-defined functions
} Env;
```

The environment is allocated on the stack in `exec_program()` and passed by pointer. Function calls copy the environment to create a local scope for parameter binding.

6.2 Symbol Table

An array of `{ name[64], double value }` entries with linear search. Capacity: 256 variables. `sym_get` exits on undefined variables; `sym_set` creates or updates.

6.3 Function Table

An array of `UserFunc` entries: function name, parameter names array, parameter count, and a pointer to the body `ExprNode`. Capacity: 64 functions. Functions can be redefined (overwrite semantics).

6.4 Expression Evaluation (`eval_expr`)

Recursive switch on `ExprKind`:

Kind	Action
`EXPR_NUM`	Return `e->num`
`EXPR_IDENT`	Look up in symbol table
`EXPR_UNOP`	Return `-eval_expr(left)`
`EXPR_BINOP`	Evaluate both sides, apply operator
`EXPR_BUILTIN`	Evaluate argument, call libm function
`EXPR_CALL`	Copy env, bind params, recurse on body

Division and modulo guard against zero; SQRT guards against negative; LOG guards against non-positive.

6.5 Condition Evaluation (`eval_cond`)

Recursive switch on `CondKind`. `COND_AND` and `COND_OR` use C short-circuit evaluation (`&&`, `||`).

6.6 Statement Execution (`exec_stmt`)

```
STMT_SET   â†’ eval_expr, sym_set
STMT_PRINT â†’ walk PrintItem list, printf with space separators
STMT_IF    â†’ eval_cond, branch to then_body or else_body
STMT_WHILE â†’ eval_cond in loop, exec_stmt(body)
STMT_FOR   â†’ evaluate from/to/step; two branches: step>0 (i<=to) or step<0 (i>=to)
STMT_DEF   â†’ func_set, print confirmation message
```

After executing each statement, `exec_stmt` walks the `s->next` pointer to execute the next statement in the list (tail recursion over the linked list).

---

7. Build Instructions

Requirements

- Flex (>= 2.6) and Bison (>= 3.0)
- GCC with `-lm`
- Linux / WSL / macOS (or Docker)

Build

```
cd MML/mml
make        # runs: yacc -d mml.y && lex mml.l && gcc y.tab.c lex.yy.c executor.c -lm -o mml
make test   # runs all 5 sample programs
make clean  # removes generated files
```

Docker (no local toolchain needed)

```
cd MML
docker build -t mml .
docker run --rm mml                          # run all 5 tests
docker run --rm -it mml ./mml               # interactive REPL
docker run --rm mml sh -c "./mml samples/test1.mml"
```

Makefile Targets

```
all:   y.tab.c lex.yy.c executor.c mml.h
       gcc -Wall -o mml y.tab.c lex.yy.c executor.c -lm

y.tab.c: mml.y
       yacc -d mml.y

lex.yy.c: mml.l y.tab.h
       lex mml.l
```

```
test:  mml
       for f in samples/test*.mml; do ./mml $$f; done

clean:
       rm -f mml y.tab.c y.tab.h lex.yy.c
```

---

8. Running Examples (with Output)

test1.mml â€” Arithmetic & Variables

```
SET a = 10
SET b = 3
PRINT "a + b =", a + b
PRINT "a / b =", a / b
PRINT "a ^ 2 =", a ^ 2
```

Output:

```
a + b = 13
a / b = 3.33333
a ^ 2 = 100
```

test2.mml â€” IF / ELSE & Boolean Logic

```
SET x = 42
IF x > 0 AND x % 2 == 0 THEN
  PRINT x, "pozitif ve cift"
ELSE
  PRINT x, "tek veya negatif"
END
```

Output:

```
42 pozitif ve cift
```

test3.mml â€” WHILE Loop (Fibonacci)

```
SET a = 1
SET b = 1
SET i = 1
WHILE i <= 10 DO
  PRINT a
  SET tmp = a + b
  SET a = b
  SET b = tmp
  SET i = i + 1
END
```

Output (first 10 Fibonacci numbers):

```
1  1  2  3  5  8  13  21  34  55
```

test4.mml â€” FOR Loop & Built-ins

```
FOR deg FROM 0 TO 360 STEP 45 DO
  PRINT deg, SIN(deg), COS(deg)
END
```

Output (selected):

```
0 0 1
45 0.850904 0.525322
90 0.893997 -0.448074
180 -0.801153 -0.59846
360 0.958916 -0.283691
```

Note: SIN/COS operate in radians internally (libm), so degrees produce non-standard values â€” this is intentional.

test5.mml â€” User-Defined Functions

```
DEF square(n) = n * n
DEF circle_area(r) = 3.14159 * square(r)
DEF max2(a, b) = (a + b + ABS(a - b)) / 2

PRINT "circle_area(5) =", circle_area(5)
PRINT "max(3, 7)      =", max2(3, 7)
```

Output:

```
[DEF] Function 'square' defined with 1 param(s)
[DEF] Function 'circle_area' defined with 1 param(s)
[DEF] Function 'max2' defined with 2 param(s)
circle_area(5) = 78.5397
max(3, 7)      = 7
```

---

9. Design Issues & Decisions

9.1 Separate `expr` and `cond` Non-Terminals

Conditions (`IF cond`, `WHILE cond`) are a distinct non-terminal from `expr`. This prevents boolean expressions from appearing in arithmetic positions (e.g., `SET x = a < b` is a syntax error). This simplifies the type system but means conditions cannot be stored in variables.

9.2 `%option case-insensitive` â€” Identifier Collision Risk

Case-insensitive matching is convenient (users can write `sin` or `SIN`), but it creates a hidden footgun: **any identifier that matches a keyword or built-in name is silently tokenized as that token**, causing a parse error rather than a variable. Affected names include: `set`, `print`, `if`, `then`, `else`, `end`, `while`, `do`, `for`, `from`, `to`, `step`, `def`, `and`, `or`, `not`, `sin`, `cos`, `tan`, `sqrt`, `abs`, `log`, `exp`, `floor`, `ceil`.

9.3 Global `arg_buf` for Call Arguments

User-defined function call arguments are accumulated in a global `arg_buf[MAX_ARGS]` / `arg_count` pair inside the parser. This works correctly for non-nested calls, and also works for simple nesting (the inner callâ€™s `call_args` fires and overwrites `arg_buf` before the outer call action reads it). A limitation is that calls with more than 16 arguments silently truncate.

9.4 Statement List as Linked List

Statements are linked via `StmtNode->next` and appended with `stmt_append()` using an O(n) walk. For programs with very many top-level statements this is quadratic. For the expected use case (script files of tens to hundreds of statements) this is acceptable.

9.5 Function Scope

Function calls create a **copy** of the entire environment (`Env local = *env`). Parameters are bound into `local.symbols`. This means functions see all global variables but cannot mutate them â€” a deliberate choice that avoids aliasing bugs. The downside is that `sizeof(Env)` is copied on every function call (array of 256 symbols plus 64 function entries, all stack-allocated).

9.6 FOR Loop with Floating-Point Step

The FOR loop variable and bounds are doubles. This allows `STEP 0.5` but also means floating-point rounding can cause the loop to execute one iteration too few or too many near the bound. For the common case of integer bounds and integer step this is not an issue.

---

10. Known Limitations

Limitation	Notes
No string variables	Strings appear only as PRINT literals
No recursion	`DEF` functions store a body `ExprNode`; no call stack is maintained for recursive calls (would loop infinitely or stack-overflow)
No `RETURN` in functions	Functions are single expressions only
No array/list type	All values are scalar doubles
No error recovery in parser	First parse error stops execution
No line-number in error messages	Flexâ€™s `yylineno` is not exposed
Case-insensitive identifier collision	See Â§9.2
`arg_buf` limit of 16 arguments	Silently truncates

---

11. Conclusion

MML demonstrates the complete Lex + Yacc + C pipeline for a non-trivial domain-specific language. The project covers:

- **Lexer design**: character class patterns, keyword/built-in disambiguation, case-insensitive matching
- **Parser design**: LALR(1) grammar, operator precedence via declarations, semantic actions building a typed AST
- **AST design**: three node types (ExprNode, CondNode, StmtNode) covering all language constructs
- **Runtime design**: recursive tree-walking executor, array-based symbol and function tables, lexical scoping for function calls

All five test programs pass without errors, demonstrating: arithmetic (test1), conditionals (test2), WHILE loops (test3), FOR loops with built-in functions (test4), and user-defined functions including nested calls (test5).

Built with Flex 2.6 + Bison 3.8 + GCC 11 on Ubuntu 22.04